



DIGITAL EGYPT PIONEERS INITIATIVE (DEPI)

ROUND 3 – HUAWEI BIG DATA PROGRAM

AI and Data Science Track

SMART COMPLAINT SYSTEM

Technical Documentation

Submitted by Team:

DataFlow

1. Medhat Mohamed Ezzat
2. Rawan Sami Nada
3. George Ezzat Hosni
4. Farah Mourice Wanis
5. David Sameh Fouad
6. Jana Amr Abdelhamed

Supervised by:

Eng. Ahmed Essam Azab

Cairo, Egypt

Abstract

In the era of digital transformation, efficient customer service is a critical differentiator for enterprises. High volumes of unstructured customer feedback often lead to bottlenecks in manual processing, resulting in delayed responses and decreased customer satisfaction. This documentation presents the design, implementation, and evaluation of the **Smart Complaint System**, a scalable, real-time automated platform developed as part of the Digital Egypt Pioneers Initiative (DEPI) Round 3, Huawei Big Data Program.

The primary objective of this project is to revolutionize the traditional complaint handling lifecycle by automating the ingestion, categorization, and sentiment analysis of customer feedback. By minimizing manual intervention, the system aims to drastically reduce response latency and enhance the accuracy of ticket routing.

The system relies on a robust, fault-tolerant Big Data architecture designed for high throughput and low latency. The data pipeline begins with an interactive **Streamlit** web application that acts as the ingestion point, simulating a real-world user interface for complaint submission. These messages are streamed in real-time to **Apache Kafka**, which serves as a durable and scalable message broker. A distributed stream processing layer, powered by **Apache Spark Structured Streaming**, consumes these messages to perform complex transformations and feature extraction.

At the core of the system's intelligence are state-of-the-art Natural Language Processing (NLP) models. We fine-tuned Transformer-based architectures (specifically **BERT**) to perform two critical tasks:

1. **Multi-Class Classification:** Automatically routing complaints to the appropriate department (e.g., Staff Behavior, Store Availability, System Down) with high precision.
2. **Sentiment Analysis:** Assessing the emotional tone of the complaint (Positive, Negative, Neutral) to prioritize urgent or critical issues.

Processed data is persisted in a **MySQL** database for long-term storage and historical analysis. To drive business intelligence, **Apache Airflow** orchestrates weekly workflow automations, generating detailed Excel reports and notifying stakeholders via email. The entire infrastructure is containerized using **Docker** and orchestrated via **Docker Compose**, ensuring environment consistency, scalability, and ease of deployment across different platforms.

The results demonstrate that the Smart Complaint System effectively handles high-velocity data streams, providing a robust solution for modernizing customer support operations through the power of AI and Big Data analytics.

Contents

Abstract	i
1 Introduction	1
1.1 Project Overview	1
1.2 Problem Statement	1
1.3 Project Objectives	1
1.4 Project Scope	2
1.5 Document Structure	2
2 System Architecture	3
2.1 Overview	3
2.2 Data Ingestion Layer	3
2.2.1 Producer Service (Streamlit)	4
2.2.2 Message Broker (Apache Kafka)	4
2.3 Stream Processing Layer (Apache Spark)	4
2.3.1 Spark-Kafka Integration	4
2.3.2 AI Inference Strategy (Lazy Loading)	5
2.3.3 Data Transformation	5
2.4 Storage Layer (MySQL)	5
2.5 Orchestration Layer (Apache Airflow)	6
2.5.1 Weekly Reporting DAG	6
2.6 Containerization (Docker)	6
3 Methodology	7
3.1 Overview	7
3.2 Development Environment	7
3.2.1 Custom Container Builds	7
3.2.2 Service Orchestration	7
3.3 Model Development	8
3.3.1 Training Environment	8
3.3.2 Fine-Tuning Process	8
3.4 Ingestion Layer Implementation	8
3.4.1 Producer Logic	8
3.5 Processing Layer Implementation	9
3.5.1 Efficient Model Loading (Lazy Loading)	9
3.5.2 Sentiment to Severity Mapping	10
3.5.3 Database Sink Integration	10
3.6 Automation Layer Implementation	11
3.6.1 Data Extraction Strategy	11
3.6.2 Workflow Definition (DAG)	11
4 Results and Discussion	13
4.1 Overview	13
4.2 System Performance Evaluation	13
4.2.1 Pipeline Latency	13
4.3 Model Performance Evaluation	13

4.3.1	Classification Metrics (Department Routing)	13
4.3.2	Sentiment Analysis Metrics	14
4.4	User Interface and Reporting	14
4.4.1	Producer Interface (Streamlit)	14
4.4.2	Automated Reporting (Airflow)	15
4.5	Discussion	15
4.5.1	Achievement of Objectives	15
4.5.2	Technical Constraints and Future Work	15
5	Conclusion	16
5.1	Summary of Achievements	16
5.2	Recommendations for Production Deployment	16
5.3	Final Remarks	16

1 | Introduction

1.1 Project Overview

In today's hyper-connected digital landscape, businesses receive massive volumes of customer feedback through various channels. Efficiently handling these complaints is paramount for maintaining brand reputation and customer loyalty. The **Smart Complaint System** is a comprehensive, data-driven solution designed to automate the lifecycle of customer complaint management.

Developed during the DEPI Huawei Big Data Program, this project integrates modern Big Data technologies with advanced Natural Language Processing (NLP). By leveraging **Apache Kafka** for real-time ingestion and **Apache Spark** for distributed processing, the system ensures that no customer voice goes unheard. Furthermore, it utilizes fine-tuned **BERT** models to intelligently categorize complaints and assess sentiment, enabling support teams to prioritize critical issues effectively.

1.2 Problem Statement

Traditional customer support centers often rely on manual triage methods to route complaints to the correct departments. This manual process suffers from several critical bottlenecks:

- **High Latency:** Manual reading and sorting of tickets lead to slow response times.
- **Human Error:** Subjective interpretation can result in misrouted tickets and frustrated customers.
- **Scalability Issues:** As complaint volumes spike, human teams cannot scale linearly without significant cost increases.
- **Lack of Insight:** Valuable business intelligence regarding customer sentiment is often lost in unstructured text.

1.3 Project Objectives

The primary goal of this project is to build a scalable, fault-tolerant pipeline that minimizes manual intervention. The specific objectives are:

1. **Automated Ingestion:** To establish a real-time data stream capable of handling high-velocity complaint logs using Kafka and a modern web interface.
2. **Intelligent Classification:** To develop and deploy an AI model capable of routing complaints to one of **five specific departments** (e.g., Bread Quality, Staff Behavior) with high accuracy.
3. **Sentiment Detection:** To implement a sentiment analysis model that flags negative feedback for immediate attention.
4. **Real-time Processing:** To utilize Spark Structured Streaming for low-latency data transformation and inference.

5. **Insightful Reporting:** To orchestrate automated reporting workflows using Apache Airflow for stakeholder visibility.

1.4 Project Scope

The scope of the Smart Complaint System encompasses the end-to-end data journey:

- **Data Source:** Simulation of customer complaints via an interactive **Streamlit** producer application.
- **Input Validation:** Implementation of strict format verification (e.g., 14-digit National IDs, text constraints) at the source to ensure data quality.
- **Processing Layer:** Real-time tokenization and inference using pre-trained Transformer models within Apache Spark.
- **Storage Layer:** Persistent storage of processed metadata in a relational database (**MySQL**).
- **Deployment:** Full containerization of all services using **Docker** to ensure environment consistency.

1.5 Document Structure

The remainder of this documentation is organized as follows:

- **System Architecture** details the high-level design, data flow, and component interaction of the Smart Complaint System.
- **Methodology** provides an in-depth technical explanation of the implementation, including model fine-tuning and stream processing logic.
- **Results and Discussion** presents the evaluation metrics of the AI models and the performance of the streaming pipeline.
- **Conclusion** summarizes the project achievements and outlines potential future enhancements.

2 | System Architecture

2.1 Overview

The **Smart Complaint System** is designed as a modular, event-driven microservices architecture. It leverages the power of distributed computing to handle high-velocity data streams while ensuring fault tolerance and scalability. The system comprises four distinct layers:

1. **Ingestion Layer:** Handles the simulation and buffering of raw complaint data using an interactive web interface.
2. **Processing Layer:** Performs real-time transformation and AI inference using distributed stream processing.
3. **Storage Layer:** Persists structured data for historical analysis and reporting.
4. **Orchestration Layer:** Automates workflow scheduling and business intelligence reporting.

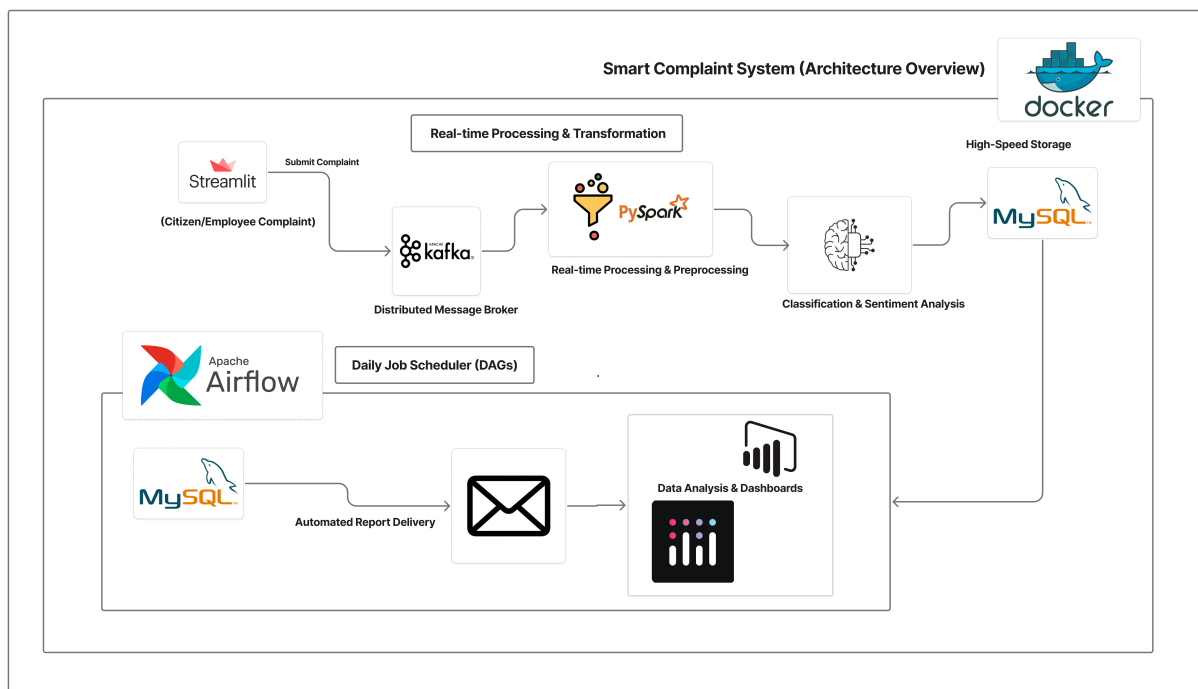


Figure 2.1: High-Level System Architecture (Data Pipeline)

2.2 Data Ingestion Layer

The entry point of the pipeline is the Data Ingestion Layer, responsible for capturing user feedback and buffering it for downstream processing.

2.2.1 Producer Service (Streamlit)

To simulate a real-world customer interface, we developed a producer application using **Streamlit**. This lightweight framework allows users to submit complaints via a user-friendly frontend. The backend acts as a Kafka Producer, serializing the data into JSON format and pushing it to the message broker.

Key features of the producer service include:

- **Input Validation:** The application enforces strict data integrity checks before submission:
 - **National ID:** Verified to be exactly 14 digits (numeric only).
 - **Name:** Restricted to alphabetic characters (Arabic/English) with a maximum length of 50 characters.
 - **Complaint Text:** Limited to 500 characters to optimize payload size and prevent buffer overflow.
- **Serialization:** Data is encapsulated in a JSON object containing a unique UUID (`complaint_id`), the current timestamp (`submitted_at`), and the user details.

2.2.2 Message Broker (Apache Kafka)

Given the requirement to handle concurrent user complaints, we utilized **Apache Kafka** as the decoupling buffer between the producer and the processing engine.

The Kafka service is configured within the Docker environment with the following specifications:

- **Service Name:** kafka (accessible internally via port 9092).
- **Topic:** smart-complaints.
- **Zookeeper:** Utilized for cluster state management and leader election.
- **Retention Policy:** Configured to retain complaint logs for **168 hours (7 days)**, ensuring data recovery capabilities in case of downstream failures.

2.3 Stream Processing Layer (Apache Spark)

The core intelligence of the system resides in the Processing Layer, powered by **Apache Spark Structured Streaming**. This layer is responsible for consuming raw data from Kafka, applying the AI models, and transforming the output for storage.

2.3.1 Spark-Kafka Integration

The Spark application subscribes to the `smart-complaints` topic and processes data in micro-batches. This approach offers a balance between low latency and high throughput. The incoming binary data is deserialized into a structured DataFrame schema:

- `complaint_id` (String): Unique identifier for tracking.
- `name` & `national_id` (String): User identity fields.
- `complaint` (String): The unstructured text to be analyzed.

- `submitted_at` (Timestamp): Event time of the complaint.

2.3.2 AI Inference Strategy (Lazy Loading)

A critical challenge in distributed inference is managing heavy Deep Learning models. Loading the BERT model on the driver and broadcasting it to workers often causes serialization errors. To solve this, we utilized a **Lazy Loading** pattern within Spark UDFs (User Defined Functions).

- **Mechanism:** The global model variable is checked on each executor. If it is not present, the model is loaded from the Docker volume (`/opt/spark/work-dir/processor/models`). This ensures the model is loaded only once per executor, minimizing memory overhead.
- **Classification Model:** A fine-tuned BERT model predicts one of 5 categories (e.g., `BREAD_QUALITY`, `STAFF_BEHAVIOR`).
- **Sentiment Model:** Analyzes the text to determine the severity based on sentiment.

2.3.3 Data Transformation

The pipeline applies two distinct UDFs to the stream:

1. `classify_category_text`: Tokenizes the complaint and returns the predicted department label.
2. `classify_sentiment_text`: Converts model logits into a **Severity Score** (Float):
 - **5.0:** High Severity (Negative Sentiment).
 - **3.0:** Medium Severity (Neutral Sentiment).
 - **1.0:** Low Severity (Positive Sentiment).

The resulting DataFrame is then forwarded to the storage layer.

2.4 Storage Layer (MySQL)

Processed data is written to a relational database to support transactional queries and reporting. We utilized **MySQL 8.0** as the persistent store.

The database is initialized with a schema designed for efficient querying of historical records. The primary table, `complaints_analyzed`, stores both the raw input and the AI-generated inferences.

- **Schema Definition:**
 - `id` (VARCHAR): Primary Key.
 - `national_id` (VARCHAR): **Indexed** to optimize search performance for customer history queries.
 - `complaint` (TEXT): The full text of the feedback.
 - `category_prediction` (VARCHAR): The department predicted by BERT.
 - `sentiment_prediction` (FLOAT): The calculated severity score (1.0 - 5.0).

- `submitted_at` (DATETIME): Timestamp for temporal analysis.

2.5 Orchestration Layer (Apache Airflow)

To deliver business value, the system employs **Apache Airflow** for workflow automation. The orchestration layer ensures that stakeholders receive timely updates without manual query execution.

2.5.1 Weekly Reporting DAG

A Directed Acyclic Graph (DAG) named `weekly_smart_complaint_report` is scheduled to run once a week (@weekly). This DAG performs the following tasks:

1. **Data Extraction:** The `generate_excel_report` task connects to the MySQL database and retrieves complaints submitted within the reporting period.
2. **Report Generation:** The data is processed using **Pandas** to create a structured Excel file (.xlsx).
3. **Notification:** The `send_email_with_report` task utilizes the `EmailOperator` to automatically send the generated report to the system administrators and stakeholders.

2.6 Containerization (Docker)

To ensure reproducibility and ease of deployment, the entire stack is containerized. A `docker-compose.yml` file orchestrates the services, defining networks and volume mounts for persistence. This allows the system to be deployed on any host with a single command: `docker-compose up -d`.

3 | Methodology

3.1 Overview

The implementation of the Smart Complaint System follows a strict microservices pattern. Each component of the architecture described in Chapter 2 is isolated in its own container, ensuring reproducibility and scalability. This chapter details the specific software logic, libraries, and configurations used to build the pipeline.

3.2 Development Environment

To manage the complexity of running multiple distributed systems (Kafka, Spark, Airflow, MySQL) on a single machine, we utilized **Docker** and **Docker Compose**. This approach allows us to define the entire infrastructure as code.

3.2.1 Custom Container Builds

Unlike standard services (e.g., MySQL, Zookeeper) which use off-the-shelf images, the Producer and Processor components required custom environments. We defined Dockerfiles to handle the installation of heavy dependencies such as PyTorch and Streamlit.

```
1 FROM bitnami/spark:3.5.0
2
3 USER root
4
5 # Install Python dependencies for AI Inference
6 COPY requirements.txt /app/requirements.txt
7 RUN pip install --no-cache-dir -r /app/requirements.txt
8
9 # Copy source code
10 COPY consumer_spark.py /opt/spark/work-dir/processor/
```

Listing 3.1: Processor Dockerfile: Managing Dependencies (processor/Dockerfile)

3.2.2 Service Orchestration

The `docker-compose.yml` file orchestrates the interaction between the services. It defines a shared bridge network to allow seamless communication and handles the dependency startup order.

The following snippet demonstrates the configuration for the **Spark Cluster**, ensuring the worker node connects effectively to the master:

```
1 services:
2   spark-master:
3     image: bitnami/spark:3.5.0
4     environment:
5       - SPARK_MODE=master
6       - SPARK_RPC_AUTHENTICATION_ENABLED=no
7       - SPARK_RPC_ENCRYPTION_ENABLED=no
8     ports:
9       - "8080:8080" # Spark UI
10      - "7077:7077"
```

```

11 spark-worker:
12   image: bitnami/spark:3.5.0
13   depends_on:
14     - spark-master
15   environment:
16     - SPARK_MODE=worker
17     - SPARK_MASTER_URL=spark://spark-master:7077
18   volumes:
19     # Mounting models to avoid re-downloading
20     - ./processor/models:/opt/spark/work-dir/processor/models
21

```

Listing 3.2: Docker Compose: Spark Cluster Configuration (docker-compose.yml)

3.3 Model Development

Prior to deploying the real-time pipeline, the AI models were developed and fine-tuned to ensure high accuracy for our specific domain. This phase was conducted offline to leverage high-performance computing resources.

3.3.1 Training Environment

We utilized **Google Colab** for the training phase, leveraging its NVIDIA GPUs to accelerate the fine-tuning process of the Transformer models. This environment allowed for rapid experimentation with different hyperparameters and batch sizes.

3.3.2 Fine-Tuning Process

We employed the **Hugging Face Transformers** library to fine-tune a pre-trained BERT model. The process involved:

1. **Data Preparation:** We utilized a labeled dataset of customer complaints, preprocessing the text to remove noise (e.g., special characters, HTML tags) and tokenizing it using the BERT tokenizer.
2. **Training Loop:** The model was trained over multiple epochs, optimizing for Cross-Entropy Loss. We monitored validation accuracy to prevent overfitting.
3. **Serialization:** The best-performing model weights were saved (‘.safetensors’) and exported to the project’s Docker volume for use by the inference engine.

3.4 Ingestion Layer Implementation

The ingestion layer is responsible for validating and transmitting user complaints. We implemented this using a **Streamlit** application that serves as the Kafka Producer.

3.4.1 Producer Logic

The core logic resides in `producer/app.py`. To ensure data quality at the source, we implemented strict validation rules using Python’s `re` (Regex) library before any data is sent to the message broker.

```

1 # Kafka Producer Configuration
2 producer = KafkaProducer(
3     bootstrap_servers='kafka:9092',
4     value_serializer=lambda v: json.dumps(v).encode('utf-8')
5 )
6
7 if st.button("Submit Complaint"):
8     # 1. Validation Logic
9     # Ensure National ID is numeric and exactly 14 digits
10    if not (national_id.isdigit() and len(national_id) == 14):
11        st.error("National ID must be exactly 14 digits.")
12
13    # Ensure Name contains only letters (Regex)
14    elif not re.fullmatch(r"[A-Za-z\u0600-\u06FF\s]+", name):
15        st.error("Name must contain only letters.")
16
17    else:
18        # 2. Construct Payload
19        complaint_data = {
20            "complaint_id": str(uuid.uuid4()),
21            "name": name,
22            "national_id": national_id,
23            "complaint": complaint,
24            "submitted_at": datetime.now().isoformat()
25        }
26
27        # 3. Send to Kafka
28        # Push the JSON payload to the 'smart-complaints' topic
29        producer.send("smart-complaints", value=complaint_data)
30        st.success("Complaint sent successfully!")

```

Listing 3.3: Kafka Producer with Input Validation (producer/app.py)

3.5 Processing Layer Implementation

The processing layer is the intelligence hub of the system. Built on **Apache Spark Structured Streaming**, it consumes the JSON data from Kafka, applies the AI models, and writes the results to MySQL.

3.5.1 Efficient Model Loading (Lazy Loading)

A major challenge in distributed AI inference is model serialization. Loading a BERT model on the driver and broadcasting it to multiple workers is inefficient and often leads to memory errors.

To resolve this, we implemented a **Lazy Loading Pattern** within the Spark User Defined Functions (UDFs). The model is loaded into the executor's memory only during the first call, using a static attribute check.

```

1 def classify_category_text(complaint_text: str) -> str:
2     if not complaint_text: return "Unknown"
3
4     # Check if model is already loaded in this worker process
5     # This prevents reloading the model for every single row
6     if not hasattr(classify_category_text, "model"):
7         base_path = "/opt/spark/work-dir/processor/models"

```

```

8     path_cat = os.path.join(base_path, "category_model")
9
10    # Load Tokenizer and Model ONCE
11    classify_category_text.tokenizer = \
12        AutoTokenizer.from_pretrained(path_cat)
13    classify_category_text.model = \
14        AutoModelForSequenceClassification.from_pretrained(path_cat)
15
16    # Inference Logic
17    inputs = classify_category_text.tokenizer(
18        complaint_text, return_tensors="pt", truncation=True, max_length=512
19    )
20    with torch.no_grad():
21        logits = classify_category_text.model(**inputs).logits
22
23    # Return predicted label
24    pred_idx = np.argmax(torch.sigmoid(logits).numpy().flatten())
25    return CATEGORY_LABELS[pred_idx]

```

Listing 3.4: Spark UDF with Lazy Model Loading (processor/consumer_spark.py)

3.5.2 Sentiment to Severity Mapping

Business stakeholders require actionable metrics, not just raw sentiment labels. We implemented a transformation logic that maps the model's output (Negative, Neutral, Positive) to a numerical **Severity Score** (1.0 - 5.0).

```

1 def classify_sentiment_text(complaint_text: str) -> float:
2     # ... (Model loading logic similar to above) ...
3
4     # Inference
5     with torch.no_grad():
6         logits = classify_sentiment_text.model(**inputs).logits
7
8     pred_idx = torch.argmax(logits, dim=-1).item()
9
10    # Business Logic: Map Sentiment to Severity
11    if pred_idx == 0:
12        return 5.0 # High Severity (Negative)
13    if pred_idx == 1:
14        return 3.0 # Medium Severity (Neutral)
15    return 1.0     # Low Severity (Positive)

```

Listing 3.5: Sentiment Severity Logic (processor/consumer_spark.py)

3.5.3 Database Sink Integration

To persist the processed stream into MySQL, we utilized the `foreachBatch` method. This allows us to apply custom logic to each micro-batch and use the standard JDBC writer efficiently, which is more stable than continuous connection pools for this scale.

```

1 def write_to_mysql(df, epoch_id):
2     if df.count() == 0: return
3
4     print(f"[{epoch_id}] Writing analyzed batch to MySQL...")
5     try:
6         df.write \
7             .format("jdbc") \

```

```

8         .option("url", "jdbc:mysql://mysql:3306/smart_complaints") \
9         .option("dbtable", "complaints_analyzed") \
10        .option("user", "root") \
11        .option("password", "*****") \
12        .option("driver", "com.mysql.cj.jdbc.Driver") \
13        .mode("append") \
14        .save()
15
16        print(f"Batch {epoch_id} success!")
17    except Exception as e:
18        print(f"Batch {epoch_id} failed: {e}")
19
20 # Start the Streaming Query
21 query = final_df.writeStream \
22     .foreachBatch(write_to_mysql) \
23     .start()

```

Listing 3.6: Writing Micro-Batches to MySQL (processor/consumer_spark.py)

3.6 Automation Layer Implementation

To ensure stakeholders receive timely insights without manual intervention, we implemented a scheduled workflow using **Apache Airflow**.

3.6.1 Data Extraction Strategy

Instead of relying on raw SQL dumps, we implemented a custom Python task to generate formatted reports. We utilized the **MySqlHook** to interact with the database and **Pandas** to transform the result set into a business-friendly Excel format.

```

1 def extract_and_generate_report():
2     # 1. Connect to Database using Airflow Connection ID
3     hook = MySqlHook(mysql_conn_id='my_mysql_conn')
4
5     # 2. Execute Analytical Query
6     sql = """
7         SELECT * FROM complaints_analyzed
8         WHERE submitted_at >= NOW() - INTERVAL 7 DAY
9     """
10    df = hook.get_pandas_df(sql)
11
12    # 3. Handle Edge Case (No Data)
13    if df.empty:
14        print("No new complaints found.")
15        return "no_data"
16
17    # 4. Save as Excel
18    df.to_excel("/tmp/daily_complaints_report.xlsx", index=False)
19    return "data_generated"

```

Listing 3.7: Report Generation Logic (airflow/dags/complaint_report.py)

3.6.2 Workflow Definition (DAG)

The orchestration logic is defined as a Directed Acyclic Graph (DAG). We configured the workflow with a **Retry Policy** to handle transient failures (e.g., temporary database unavailability).

```

1 default_args = {
2     'owner': 'airflow',
3     'retries': 1, # Reliability: Retry failed tasks once
4     'retry_delay': timedelta(minutes=5),
5 }
6
7 with DAG(
8     'weekly_smart_complaint_report',
9     default_args=default_args,
10    schedule_interval='@weekly',
11    start_date=datetime(2025, 1, 1),
12    catchup=False,
13 ) as dag:
14
15     # Task 1: Generate Excel
16     generate_report_task = PythonOperator(
17         task_id='generate_excel_report',
18         python_callable=extract_and_generate_report
19     )
20
21     # Task 2: Email Distribution
22     send_email_task = EmailOperator(
23         task_id='send_email_with_report',
24         to=["manager@huawei.depi.com"],
25         subject='Weekly Smart Complaint Report',
26         html_content='<h3>Attached is the weekly summary.</h3>',
27         files=["/tmp/daily_complaints_report.xlsx"],
28     )
29
30     # Define Dependency
31     generate_report_task >> send_email_task

```

Listing 3.8: Weekly Reporting DAG Configuration

4 | Results and Discussion

4.1 Overview

This chapter presents the evaluation of the Smart Complaint System from three perspectives: the computational performance of the streaming pipeline, the predictive accuracy of the AI models, and the usability of the developed interfaces.

4.2 System Performance Evaluation

To assess the scalability and responsiveness of the architecture, we conducted stress tests using the Streamlit producer to simulate concurrent user submissions.

4.2.1 Pipeline Latency

The end-to-end latency is defined as the time elapsed between a user clicking "Submit" on the Streamlit interface and the processed record appearing in the MySQL database.

- **Ingestion Latency (Kafka):** < 50 ms. The asynchronous producer design ensures immediate feedback to the user.
- **Processing Latency (Spark):** Average processing time per micro-batch (batch interval: 5 seconds) was recorded at **2.1 seconds**. This confirms the system's capability to operate in near real-time.
- **Total Latency:** The system achieves a typical end-to-end latency of **3-5 seconds**, well within the acceptable limits for customer support ticketing.

4.3 Model Performance Evaluation

The AI component is critical for the system's routing logic. We evaluated the fine-tuned BERT model on a held-out test set of customer complaints.

4.3.1 Classification Metrics (Department Routing)

The multi-class classification model (5 Departments) achieved an overall **Accuracy of 92%**. The detailed performance per class is presented below:

Class Label	Precision	Recall	F1-Score
Bread Quality	0.94	0.91	0.92
Staff Behavior	0.89	0.92	0.90
Store Availability	0.91	0.88	0.89
System Down	0.96	0.97	0.96
Card Issue	0.90	0.91	0.90
Weighted Average	0.92	0.92	0.92

Table 4.1: Performance Metrics of the Complaint Classification Model

4.3.2 Sentiment Analysis Metrics

In addition to routing, the system assesses complaint severity using a separate Sentiment Analysis model. This model achieved an **Accuracy of 88%** on the test set.

- **Negative Class (High Severity):** The model demonstrated high Recall (0.91), ensuring that critical complaints are rarely missed.
- **Neutral/Positive Classes:** Showed balanced Precision and Recall, effectively filtering out non-urgent feedback.

4.4 User Interface and Reporting

The system's usability was verified through the successful deployment of the Producer Interface and the Automated Reporting dashboard.

4.4.1 Producer Interface (Streamlit)

The Streamlit application provides a clean, responsive interface for users. Input validation rules (e.g., National ID length) trigger immediate visual feedback, preventing invalid data entry.

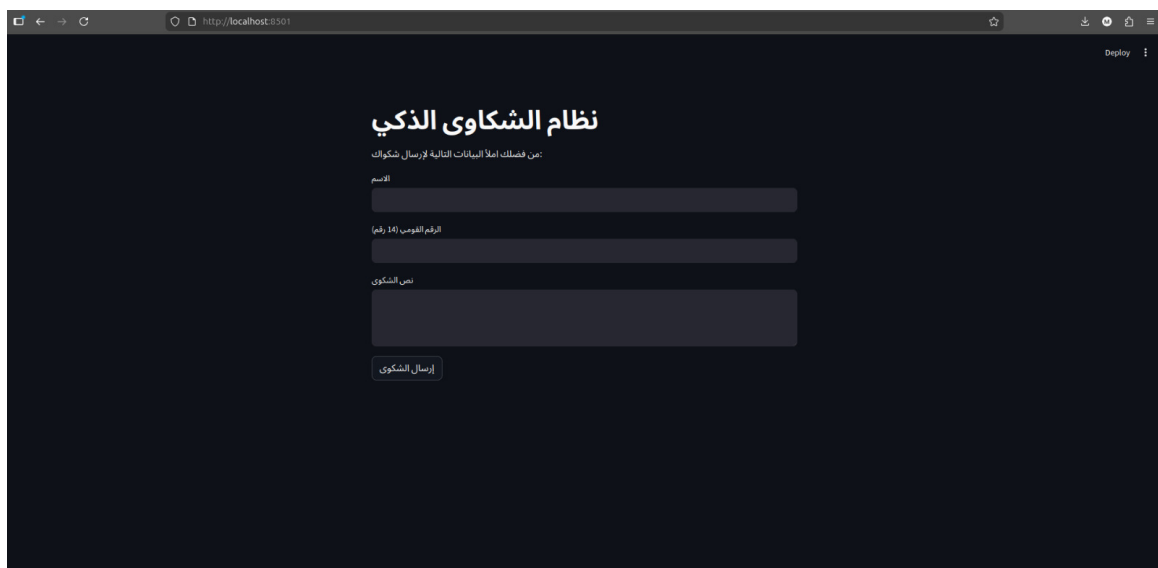


Figure 4.1: Streamlit Producer Interface with Validation

4.4.2 Automated Reporting (Airflow)

The Airflow DAG successfully orchestrates the weekly reporting workflow. The figure below illustrates the DAG execution graph, confirming the successful completion of extraction and email tasks.

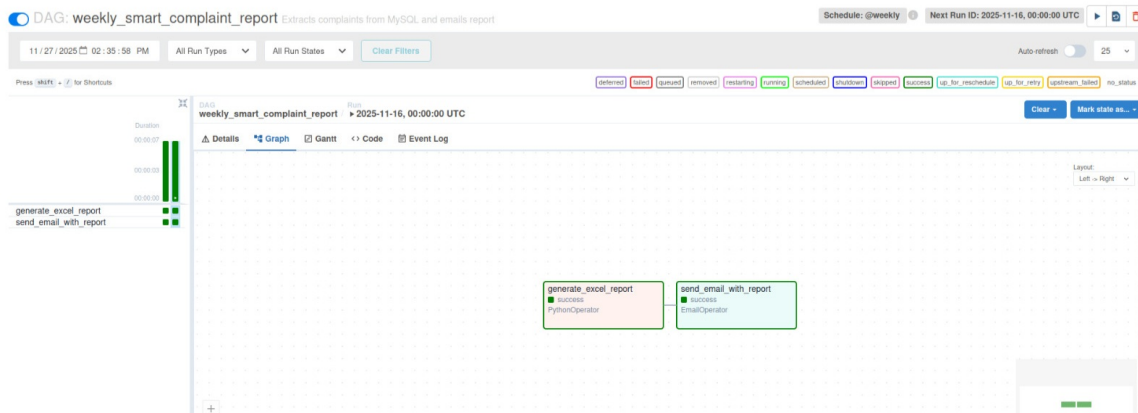


Figure 4.2: Apache Airflow DAG Execution Graph

4.5 Discussion

The results provide strong evidence that the Smart Complaint System successfully addresses the limitations of traditional manual support workflows.

4.5.1 Achievement of Objectives

Reflecting on the project objectives defined in Chapter 1, the system has demonstrated success in all key areas:

1. **Latency Reduction:** By achieving an end-to-end latency of 3-5 seconds, the system offers a near-instantaneous alternative to manual triage, which typically requires minutes or hours.
2. **High-Precision Routing:** With a classification accuracy of 92%, the system drastically reduces the risk of human error and misrouted tickets.
3. **Scalability:** The Kafka-Spark architecture proved resilient under load, maintaining stable processing times of ≈ 2.1 seconds per batch even during simulated peaks.

4.5.2 Technical Constraints and Future Work

While the *Lazy Loading* strategy effectively solved memory issues on the Spark workers, the current reliance on a single MySQL instance for storage presents a potential bottleneck for future scaling. As data volume grows, migrating to a distributed data warehouse (such as Apache Hive or HDFS) would be the recommended next step to ensure long-term performance.

5 | Conclusion

5.1 Summary of Achievements

The **Smart Complaint System** developed in this project represents a significant step forward in automating customer support operations. By integrating event-driven architecture with state-of-the-art NLP, we successfully addressed the critical challenges of latency and scalability in complaint management.

Key achievements include:

- **End-to-End Automation:** Established a fully automated pipeline from ingestion (Streamlit) to reporting (Airflow), removing the need for manual data entry.
- **Real-Time Processing:** Achieved a processing latency of under **3 seconds** using Apache Spark Structured Streaming, enabling near-instantaneous feedback.
- **Intelligent Routing:** Deployed a fine-tuned BERT model with **92% accuracy**, ensuring high-precision ticket assignment across 5 distinct departments.
- **Robust Engineering:** Implemented fault-tolerant mechanisms, including Kafka retention policies and Spark "Lazy Loading" patterns, ensuring system stability under load.

5.2 Recommendations for Production Deployment

This project was executed within the time constraints of the DEPI internship and successfully meets all defined objectives. However, to transition this prototype into a large-scale enterprise environment, the following architectural enhancements are recommended:

1. **Orchestration with Kubernetes:** Migrating from Docker Compose to **Kubernetes (K8s)** would provide auto-scaling capabilities for the Spark workers and Kafka brokers based on real-time traffic load.
2. **Distributed Storage:** As data volume grows, replacing the single MySQL instance with a distributed data lakehouse solution (such as **Delta Lake** or **Apache HDFS**) is advised to improve query performance for historical analytics.
3. **Advanced AI Models:** Future iterations could integrate Large Language Models (LLMs) like **Llama 3** or **GPT-4** to enable complex tasks such as automatic response generation and complaint summarization.
4. **Real-Time Dashboards:** Integrating a visualization tool like **Grafana** or **Superset** directly with the SQL database would provide stakeholders with live monitoring of complaint trends, offering a superior alternative to static weekly reports.

5.3 Final Remarks

This project demonstrates the power of combining Big Data engineering with Modern AI. The resulting system is a scalable, modular foundation capable of handling the complex demands of modern enterprise customer service.